

Projeto RAG Multimodal - Documento de Teste Maior

Arquivo criado para validar ingestão de DOCX e PDF com o mesmo conteúdo em um pipeline de RAG.

1. Visão geral

Este documento foi criado para testar um projeto de RAG multimodal com arquivos PDF, DOCX e imagens. O objetivo é verificar se o sistema consegue ingerir documentos maiores, dividir o conteúdo em chunks, gerar embeddings, salvar em um banco vetorial e recuperar as informações corretas durante uma conversa.

O projeto usa Python, OpenAI, ChromaDB e loaders específicos para cada tipo de arquivo. A ingestão transforma cada fonte em texto pesquisável. Em seguida, o texto é dividido em partes menores para melhorar a busca semântica.

Este mesmo conteúdo existe em dois formatos: DOCX e PDF. Isso permite comparar se a extração de texto do PDF e do DOCX gera resultados semelhantes no pipeline de RAG.

2. Personagem de teste

Sydney Sweeney é usada aqui como entidade principal de teste. No contexto deste documento, ela é descrita como uma atriz norte-americana conhecida por seus papéis nas séries Euphoria e The White Lotus.

O documento também menciona o filme Anyone But You como um exemplo de filme associado a Sydney Sweeney. Essa informação deve ser recuperada quando o usuário perguntar qual filme foi mencionado.

A nacionalidade mencionada para Sydney Sweeney neste documento é Estados Unidos. O objetivo não é criar uma biografia completa, mas oferecer dados suficientes para testar perguntas factuais, recuperação por entidade e respostas com citações.

3. Imagem de apoio

A imagem usada no teste contém o texto HOW WELL DOES SYDNEY SWEENEY KNOW HER IMDb PAGE?. Ela serve para validar se o image loader consegue transformar conteúdo visual em um documento textual pesquisável.

Quando o usuário pergunta o que foi perguntado na imagem, o sistema deve recuperar a fonte da imagem e responder que a pergunta exibida foi HOW WELL DOES SYDNEY SWEENEY KNOW HER IMDb PAGE?.

A descrição visual da imagem também pode incluir uma mulher sorridente, fundo escuro, roupa branca e estilo promocional. Esses detalhes ajudam em buscas semânticas sobre o conteúdo visual.

4. Pipeline de ingestão

O pipeline de ingestão começa lendo os arquivos dentro da pasta data/raw. Arquivos PDF são processados pelo pdf_loader, arquivos DOCX pelo docx_loader e imagens JPG ou PNG pelo image_loader.

Depois da leitura, todos os documentos viram texto. Esse texto passa pelo chunker, que divide o conteúdo em blocos menores. Cada chunk recebe metadados como source, file_type, chunk_index e indexed_at.

Os chunks são enviados para o modelo de embeddings text-embedding-3-small. O vetor gerado é salvo no ChromaDB junto com o texto original e os metadados.

5. Busca vetorial

Na consulta, a pergunta do usuário também vira embedding. O sistema usa esse vetor para buscar os chunks mais próximos no ChromaDB.

As distâncias retornadas indicam proximidade semântica. Em geral, quanto menor a distância, mais parecido é o chunk com a pergunta. Porém, distâncias pequenas não garantem que o chunk seja sempre o melhor para responder.

Por isso o projeto usa re-ranking. O Chroma retorna candidatos e o reranker escolhe os trechos mais relevantes considerando a pergunta e o texto de cada chunk.

6. Re-ranking

O re-ranking melhora a qualidade da recuperação porque embeddings são bons para encontrar candidatos, mas nem sempre ordenam perfeitamente os melhores trechos.

Neste projeto, o reranker recebe a pergunta e os chunks candidatos. Ele retorna os índices dos chunks mais relevantes. Depois, apenas esses chunks entram no contexto final enviado ao modelo de linguagem.

Esse desenho reduz ruído e evita que documentos parcialmente relacionados contaminem a resposta final.

7. Memória conversacional

O projeto também usa reescrita de perguntas para suportar conversas. Se o usuário pergunta Quem é Sydney Sweeney? e depois pergunta Qual série ela fez?, o sistema deve entender que ela se refere a Sydney Sweeney.

A etapa question_rewriter recebe o histórico recente e a pergunta atual. O modelo reescreve a pergunta para que ela fique completa e independente do histórico.

Esse padrão é comum em RAG conversacional, pois a busca vetorial funciona melhor quando a pergunta contém todos os termos importantes.

8. Citações

As respostas do sistema devem citar as fontes usadas. O contexto enviado ao modelo inclui referencias como [1], [2] e [3], associadas a source e chunk_index.

Ao final da resposta, o usuario consegue ver quais chunks foram usados e quais distancias foram retornadas. Isso melhora a auditabilidade do RAG.

Em um sistema corporativo, citacoes ajudam usuarios a confiar na resposta e permitem investigar rapidamente se a recuperacao trouxe o documento correto.

9. Avaliacao

O projeto possui um conjunto de testes em tests/test_cases.py. Cada caso define pergunta, palavras esperadas, fontes esperadas, fontes esperadas no retrieval e fontes esperadas no rerank.

A avaliacao mede Retrieval, Rerank, Answer e Overall. Isso permite identificar se uma falha aconteceu na busca vetorial, no reranking ou na geracao da resposta.

Um exemplo de teste negativo e perguntar pela data de nascimento de Sydney Sweeney. Como este documento nao informa essa data, o sistema deve responder que nao encontrou essa informacao nos documentos.

10. Perguntas esperadas

Pergunta: Quem e Sydney Sweeney? Resposta esperada: Sydney Sweeney e uma atriz norte-americana conhecida por Euphoria e The White Lotus.

Pergunta: Quais series sao mencionadas? Resposta esperada: Euphoria e The White Lotus.

Pergunta: Qual filme e mencionado? Resposta esperada: Anyone But You.

Pergunta: O que perguntaram na imagem? Resposta esperada: HOW WELL DOES SYDNEY SWEENEY KNOW HER IMDb PAGE?.

Pergunta: Qual e a data de nascimento de Sydney Sweeney? Resposta esperada: Nao encontrei essa informacao nos documentos.

11. Proximas melhorias

As proximas melhorias tecnicas incluem metadata filtering, multi-collections, hybrid search, parent-child retrieval, cache de embeddings e observabilidade de custo e latencia.

Metadata filtering permite restringir a busca por file_type, source ou document_type. Multi-collections permite separar dominios como documents, images, contracts e manuals.

Hybrid search combina busca lexical e busca vetorial. Parent-child retrieval permite buscar em chunks pequenos, mas responder com um contexto maior do documento pai.

12. Conclusao

Este documento maior foi criado para testar um RAG com mais volume de texto do que exemplos pequenos. Ele deve gerar varios chunks e permitir uma avaliacao mais realista do pipeline.

Como o mesmo conteudo esta disponivel em DOCX e PDF, o sistema pode comparar a qualidade da ingestao entre formatos diferentes.

Se o RAG estiver funcionando corretamente, ele deve recuperar informacoes sobre Sydney Sweeney, as series Euphoria e The White Lotus, o filme Anyone But You, a pergunta presente na imagem e tambem recusar perguntas que nao tenham resposta nos documentos.

13. Cenario corporativo

Em um ambiente corporativo, o mesmo padrao poderia ser aplicado a contratos, politicas internas, manuais tecnicos, chamados de suporte e documentos de onboarding.

Cada documento receberia metadados padronizados. Isso permitiria perguntas como buscar apenas documentos PDF, buscar apenas imagens ou buscar apenas arquivos de uma determinada area.

O pipeline precisa ser observavel. Logs de retrieval, distancias, fontes, chunks selecionados e custo por chamada ajudam a entender o comportamento do sistema.

14. Riscos comuns

Um risco comum em RAG e o falso negativo. Isso acontece quando a resposta existe, mas o sistema nao recupera o chunk correto. Outro risco e o falso positivo, quando o sistema usa um contexto relacionado, mas insuficiente, e gera uma resposta indevida.

Threshold de distancia pode ajudar a reduzir ruido, mas tambem pode eliminar chunks relevantes. Por isso, neste projeto, o threshold deve ser configuravel e preferencialmente usado como sinal de confianca, nao como regra rigida.

A avaliacao por etapa ajuda a diagnosticar esse tipo de problema. Se o retrieval falha, devemos revisar chunking, embeddings ou top_k. Se o rerank falha, devemos revisar o prompt ou o modelo. Se a resposta falha, devemos revisar o prompt de geracao.

15. Resumo tecnico

Modelos usados no projeto: gpt-4o-mini para leitura, resposta e reranking; text-embedding-3-small para embeddings.

Banco vetorial usado: ChromaDB com armazenamento persistente local em chroma_db. Dados brutos ficam em data/raw.

Arquivos suportados: PDF, DOCX, JPG, JPEG e PNG. O suporte a imagem usa modelo multimodal para gerar um texto descritivo indexavel.